

Tarea Sesión 13 — RAG con Chroma sobre el Agente Personal

Versión RAG de `Tarea_Agente_Personal` (Sesión 12): el mismo agente de planificación académica,

con `buscar_en_documentos` reemplazada por **búsqueda semántica real** (Chroma + embeddings),

en vez de la búsqueda de texto literal de la versión anterior. Ver `ENTREGA_GOOGLE_DOC.md` para la descripción completa (caso personal, arquitectura, código) — este README es solo la guía de

setup para correrlo.

Requisitos previos

- Un entorno Python con `langchain`, `langchain-anthropic`, `langchain-openai`, `langchain-ollama`, `python-dotenv`, `python-docx`, `pypdf` ya instalados (este proyecto reutiliza el venv de `Modulo4_Agentes_Cognitivos/.venv` — no crea uno propio).
- [Ollama](#) instalado y corriendo localmente (se usa para generar los embeddings del RAG, sin depender de una API de pago).
- (Opcional) LM Studio corriendo local si usas `AGENT_MODEL=gemma-lmstudio`.
- (Opcional) Cuenta gratuita en [tavily.com](#) si quieres la búsqueda web de respaldo (`buscar_en_la_web`).

1. Instalar lo que falte en el venv

Este proyecto **no crea un venv nuevo**. Instala únicamente los paquetes que ese entorno todavía

no tenía (Chroma y afines):

```
Modulo4_Agentes_Cognitivos/.venv/bin/pip install -r
Tarea_Agente_Personal_RAG/requirements.txt
```

`pip` no reinstala ni toca `langchain` / `langchain-anthropic` / `langchain-openai` / `langchain-ollama`

si ya están (los deja como están) — solo agrega lo que falta: `langchain-chroma`, `chromadb`, `langchain-text-splitters`, `pydantic-settings`, `tavily-python`.

2. Modelo de embeddings de Ollama

```
ollama pull nomic-embed-text
```

Se descarga una sola vez (~274 MB). Si tu servidor Ollama no corre en `localhost:11434` , ajusta `OLLAMA_BASE_URL` en `.env` .

3. Configurar `.env`

Ya viene completo con las mismas keys que `Tarea_Agente_Personal` (Sesión 12):
`ANTHROPIC_API_KEY` ,
`AGENT_MODEL` (por defecto `gemma-lmstudio`), `LMSTUDIO_*` , `LANGSMITH_*` (proyecto propio: `tarea-agente-rag-m5s13`). Solo agrega tu `TAVILY_API_KEY` si quieres la búsqueda web de respaldo — sin ella, `buscar_en_la_web` responde con un mensaje claro en vez de fallar.

4. Correr el agente

```
python agente_planificacion_rag.py
```

En el primer arranque construye el índice vectorial en `chroma_index/` a partir de todo lo que haya en `materiales/` (puede tardar unos segundos por documento, según cuántos haya). Las siguientes corridas cargan el índice ya construido, sin reconstruirlo.

Para forzar reconstruir el índice (por ejemplo, después de agregar documentos nuevos a `materiales/`):

```
python agente_planificacion_rag.py --reindexar
```

Solución de problemas

Síntoma	Causa	Solución
<code>ConnectionError</code> al indexar o buscar	Ollama no está corriendo	Arráncalo (<code>ollama serve</code> , o abre la app) y confirma con <code>curl http://localhost:11434/api/tags</code> .
<code>buscar_en_documentos</code> no encuentra nada aunque el archivo existe	El índice se construyó antes de agregar ese archivo a <code>materiales/</code>	Corre <code>python agente_planificacion_rag.py --reindexar</code> .

Síntoma	Causa	Solución
<code>buscar_en_la_web</code> responde "Busqueda web no disponible"	Falta <code>TAVILY_API_KEY</code> en <code>.env</code>	Genera una cuenta gratuita en tavily.com y pega la key — es opcional, el resto del agente funciona igual sin ella.